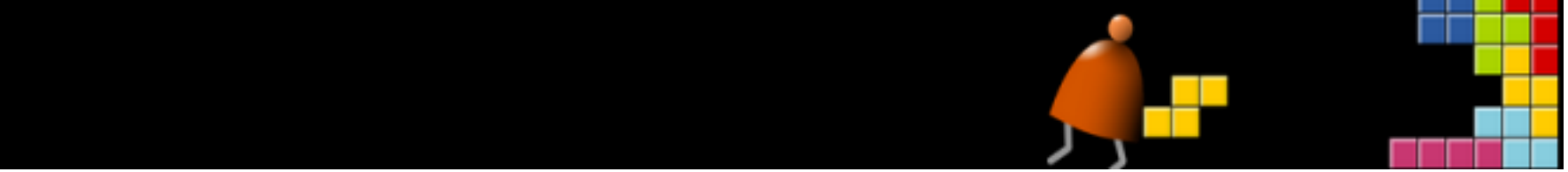


From Nand to Tetris

Building a Modern Computer from First Principles

Lecture 1

Boolean Logic

Slide deck for Chapter 1 of the book

The Elements of Computing Systems (2nd edition)

By Noam Nisan and Shimon Schocken

MIT Press

Chapter 1: Boolean logic

Theory

• Basic concepts • Nand • Logic gates • HDL • Hardware simulation • Multi-bit buses

Practice

Project 1

• Introduction • Chips • Guidelines

Boolean values

Boolean values

Boolean values

off on no

```
          false
yes                0 1
          true
```

Different labels, all referring to two possible states.

George Boole 1815 - 1864

Boolean values

$b_1 b_0$

1 binary variable: 2 possible states

Boolean values

$b_1 b_0$

1 binary variable: 2 possible states

2 binary variables: 4 possible states

Boolean values

$b_2\, b_1 b_0$

1 binary variable: 2 possible states

2 binary variables: 4 possible states

Boolean values

$\ldots\ b_1 b_0$

b_2

1 binary variable: 2 possible states

2 binary variables: 4 possible states

3 binary variables: 8 possible states

. . .

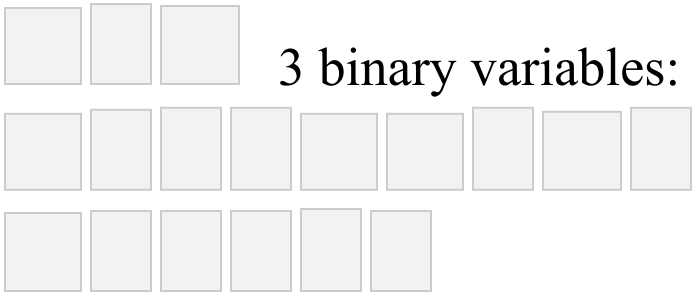
Boolean values

$b_2\, b_1 b_0$

...

1 binary variable: 2 possible states

2 binary variables: 4 possible states



3 binary variables: 8 possible states

...

Question: How many different states can be represented by N binary variables?

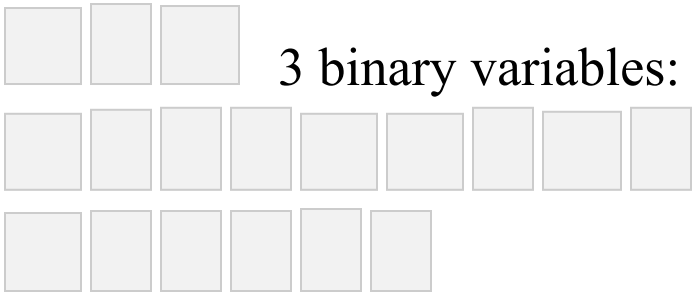
Boolean values

$b_2\, b_1 b_0$

...

1 binary variable: 2 possible states

2 binary variables: 4 possible states



3 binary variables: 8 possible states

states can be represented by N binary variables?

...

Answer: 2^N

Question: How many different

Boolean functions

x y	f
0 0	0
0 1	0
1 0	0
1 1	1

Boolean functions

x y	f
0 0	0

0 1	0
1 0	0
1 1	1

Boolean functions

x y	f
0 0	0
0 1	0
1 0	0
1 1	1

x $f(x,y)$

y f

$$f(x,y) = \begin{cases} 1 & \text{when } x == 1 \text{ and } y == 1 \\ 0 & \text{otherwise} \end{cases}$$

Boolean functions

x y	And
0 0	0
0 1	0
1 0	0
1 1	1

y

And(x,y) =

Boolean function
And

x And (x, y)

otherwise

`1` when x `==` `1` and y `==` `1` `0`

A function that operates on boolean variables, and returns a boolean value

Simple boolean functions (like And):

- Sometimes called *operators*
- Their variables are called *operands*
- ��(��, ��) can also be written as �� �� ��

Boolean functions

x And y

x y	And
`0 0`	`0`
`0 1`	`0`

1 0	0
1 1	1

Boolean functions

x And y x Or y

x y	And
0 0	0
0 1	0
1 0	0
1 1	1

x y	Or
0 0	0
0 1	1
1 0	1
1 1	1

Boolean functions

x And y x Or y Not(x)

x y	And
0 0	0

0 1	0
1 0	0
1 1	1

0 0	0
0 1	1
1 0	1
1 1	1

x	Not
0	1
1	0

x y	Or

Boolean functions

x And y x Or y Not(x)

x y	And
0 0	0
0 1	0
1 0	0
1 1	1

x y	Nand
0 0	1
0 1	1
1 0	1
1 1	0

0 1	1
1 0	1
1 1	1

x	Not
0	1
1	0

x Nand y

x y	Or
0 0	0

Boolean functions

x And *y*

x *y*	And
0 0	0
0 1	0
1 0	0
1 1	1

x Or *y*

x *y*	Or
0 0	0
0 1	1
1 0	1
1 1	1

Not(*x*)

0 1	1
1 0	1
1 1	0

x Nand *y*

x *y*	Nand
0 0	1

x Xor *y*

x *y*	Xor
0 0	0
0 1	1
1 0	1
1 1	0

x	Not
0	1
1	0

Boolean functions

x And *y* *x* Or *y*

Not(*x*)

x y	And
0 0	0
0 1	0
1 0	0
1 1	1

x y	Or
0 0	0
0 1	1
1 0	1
1 1	1

x	Not
0	1
1	0

x Nand y x f yx Xor y

x y	Nand
0 0	1
0 1	1
1 0	1
1 1	0

x y	f
0 0	
0 1	
• • •	
1 0	
1 1	

0 0		v_1
0 1		v_2
1 0		v_3
1 1		v_4

Boolean functions

x And y x Or y

Question:

x y	And
0 0	0
0 1	0
1 0	0
1 1	1

x Nand y

x y	Or
0 0	0
0 1	1
1 0	1
1 1	1

How many Boolean functions $x\ f\ y$ exist over two binary (2-valued) variables?

Answer: 16

N binary variables span 2^{2^N} Boolean functions.

x Xor y

x y	Nand
0 0	1
0 1	1
1 0	1
1 1	0

$x\ f\ y$

x y	f
0 0	v_1
0 1	v_2
1 0	v_3
1 1	v_4

Boolean functions

x And y x Or y

Not(x)

x y	And
0 0	0
0 1	0
1 0	0
1 1	1

x y	O
0 0	0
0 1	1
1 0	1
1 1	1

y	Not
0	1
1	0

Boolean function evaluation (example):

Boolean functions

x And *y* *x* Or *y*

Not(*x*)

x y	And
0 0	0

0 1	0
1 0	0
1 1	1

0 0	0
0 1	1
1 0	1
1 1	1

y	Not
0	1
1	0

x y	O

Boolean function evaluation (example):

Not(x Or (y And z)) Evaluate this function for, say, $x = 0$, $y = 1$, $z = 1$

Not(0 Or (1 And 1)) =

Not(0 Or 1) =

Not(1) =

0

Chapter 1: Boolean logic

Theory

• Basic concepts • Nand

Practice

• Logic gates

Multi-bit buses

• HDL

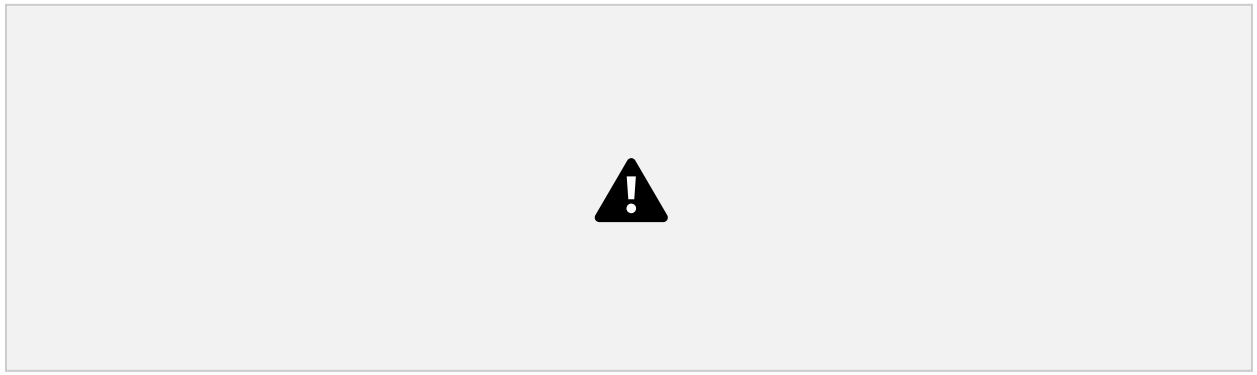
Project 1

• Hardware simulation •

• Introduction • Chips

• Guidelines

The expressive power of Nand



Observations

- Not(x) = x Nand x

- Not can be realized using Nand

Thus:

- x And y = Not(x Nand y)
- And can be realized using Nand
- x Or y = Not(Not(x) And Not(y))
- Or can be realized using Nand

(De Morgan)

Theorem: Any Boolean function can be realized using only Nand.

Proof : Any Boolean function can be expressed as a truth table. Any truth table can be expressed as a Boolean function using only Not, And, and Or (synthesized as a DNF, Disjunctive Normal Form). Combined with the above observations, Q.E.D.

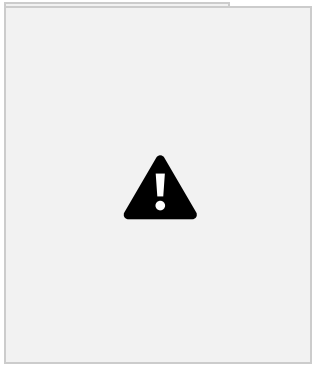
The expressive power of Nand

Theorem: Any Boolean function can be realized using only Nand.

The expressive power of Nand

Theorem: Any Boolean function can be realized using only Nand.

Implication: Any computer can be built from Nand gates only:



Computers:
Machines that realize Boolean functions:

f (input bits) = output bits

OK, so we *can* build a computer from Nand gates only. **But… how can we *actually* do it?**

That's what the Nand to Tetris course is all about!

Chapter 1: Boolean logic

Theory • Basic concepts

• Nand

Practice

• Logic gates

• HDL

• Hardware simulation •

Multi-bit buses

Project 1

• Introduction • Chips

• Guidelines

Chapter 1: Boolean logic

Theory

• Basic concepts • Nand to Tetris

Practice

• Logic gates

• HDL

• Hardware simulation • Multi-bit buses

Project 1

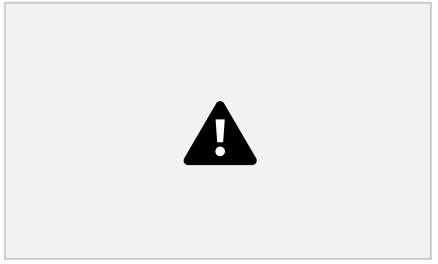
• Introduction • Chips

• Guidelines

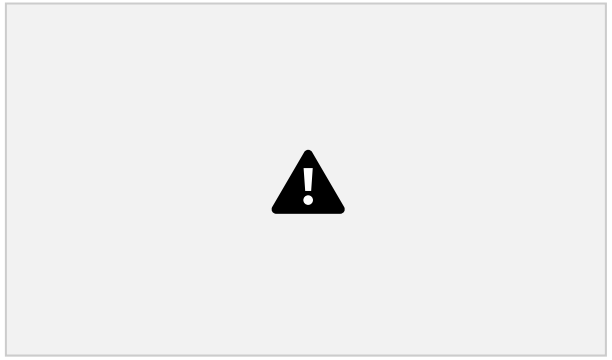
Logic gates

- Elementary gates (`Nand`, `And`, `Or`, `Not`, ...)
- Composite gates (`Mux`, `Adder`, ...)

Elementary gates



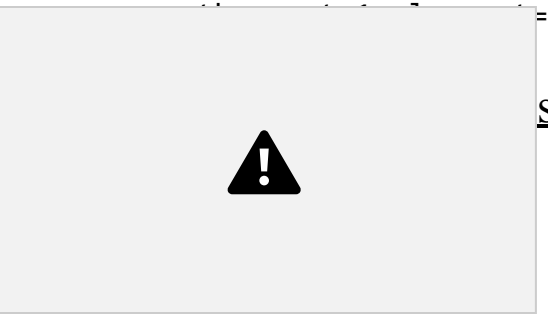
```
if (a==1 and b==1)
then out=0 else out=1
```



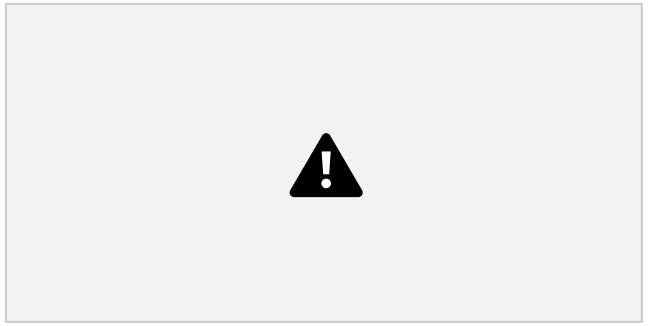
if

```
(a==1 or b==1)
```

```
then out=1 else out=0
```



se particular gates?



```
if (in==0)
then out=1 else out=0
```

And

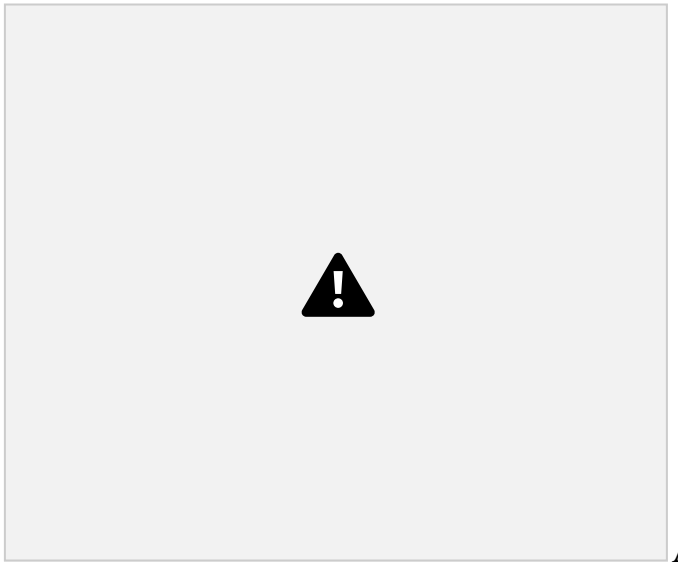
```
if (a==1 and b==1)
```

• Because either {`Nand`} or {`And`, `Or`, `Not`} (as well as other subsets of Boolean operators) can be used to span any given Boolean function • Because they have efficient and direct hardware implementations.

Elementary gates

And `if (a==1 and b==1)`

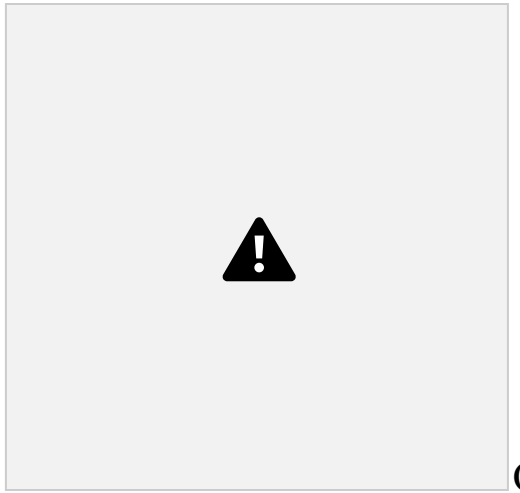
```
then out=1 else out=0
```



```
if (a==1 or b==1) then out=1
else out=0
```

And Circuit implementations (conceptual):

circuit



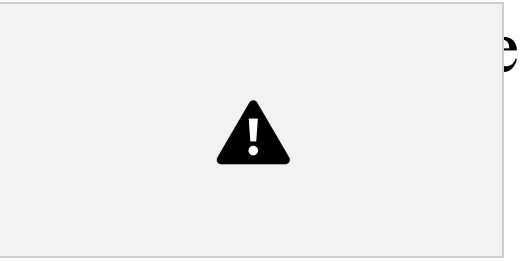
Or circuit

e gates

```
if (a==1 and b==1 and
c==1) then out=1 else
        out=0
```

And3

e gates



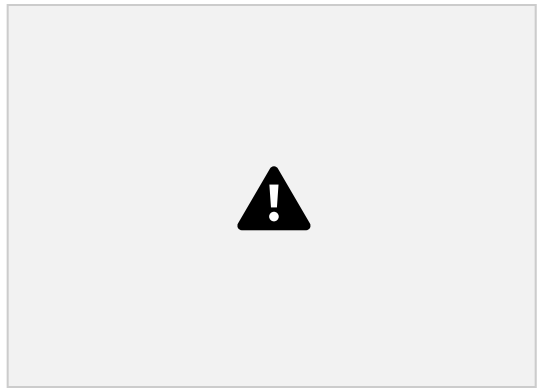
And3

```
c==1) then out=1 else
        out=0
```

Possible implementations:

```
if (a==1 and b==1 and
```

Physical Logical



• This course does not deal with physical implementations (circuits, transistors,... that's EE, not CS)

• We'll focus on logical implementations.

Theory

• Basic concepts • Nand to Tetris

Practice

• Logic gates

• HDL

• Hardware simulation •

Multi-bit buses

Project 1

• Introduction • Chips

• Guidelines

Building a chip

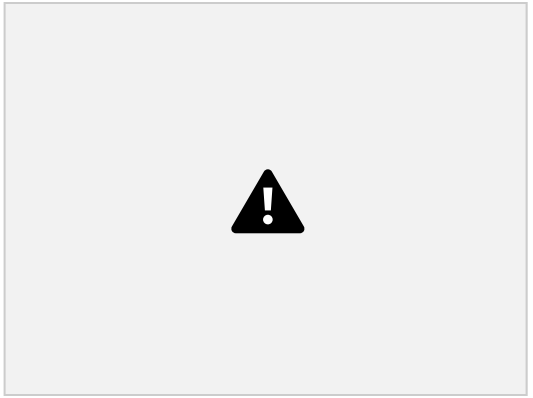
b

a

Xor

out

if ((a == 0 and b == 1) or (a == 1 and b == 0)) out = 1
else
out = 0



(an arbitrary image found on the Internet...)

The process

- Design the chip architecture
- Specify the architecture in HDL • Test the chip in a hardware simulator •

Optimize the design

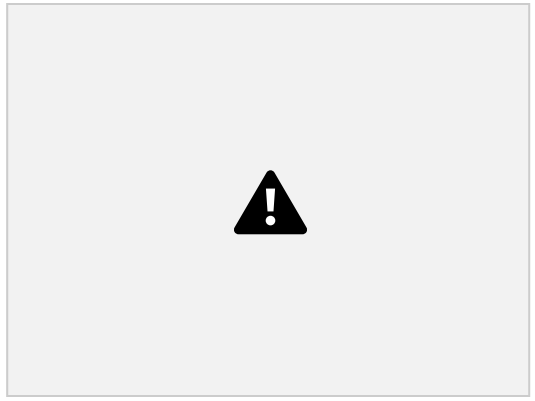
- Realize the optimized design in silicon.

Building a chip

a

Xor

b



out

```
if ((a == 0 and b == 1) or (a == 1 and b
== 0)) out = 1
else
 out = 0
```

The process

(an arbitrary image found on the Internet...)

✓ Design the chip architecture

✓ Specify the architecture in HDL ✓

Test the chip in a hardware simulator •

Optimize the design

- Realize the optimized design in silicon.

Design:

Requirements

Xor

a b out

0 0 0

1 1 1

Requirement Build a chip that delivers this functionality

if ((a == 0 and b == 1) or (a == 1 and b == 0))

out = 1

else

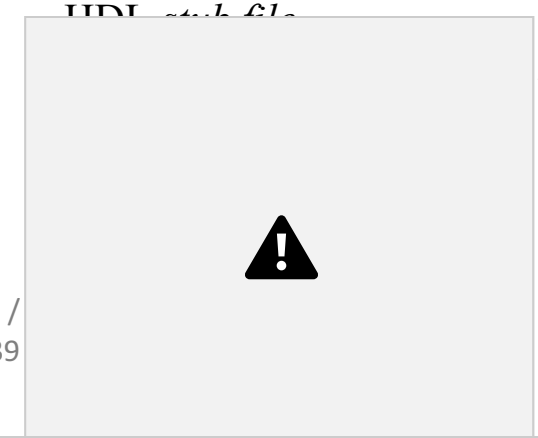
out = 0

```
Or (Not(a) And b)) */ CHIP Xor {
 IN a, b;
 OUT out;
 PARTS:
 // Missing implementation
}
```

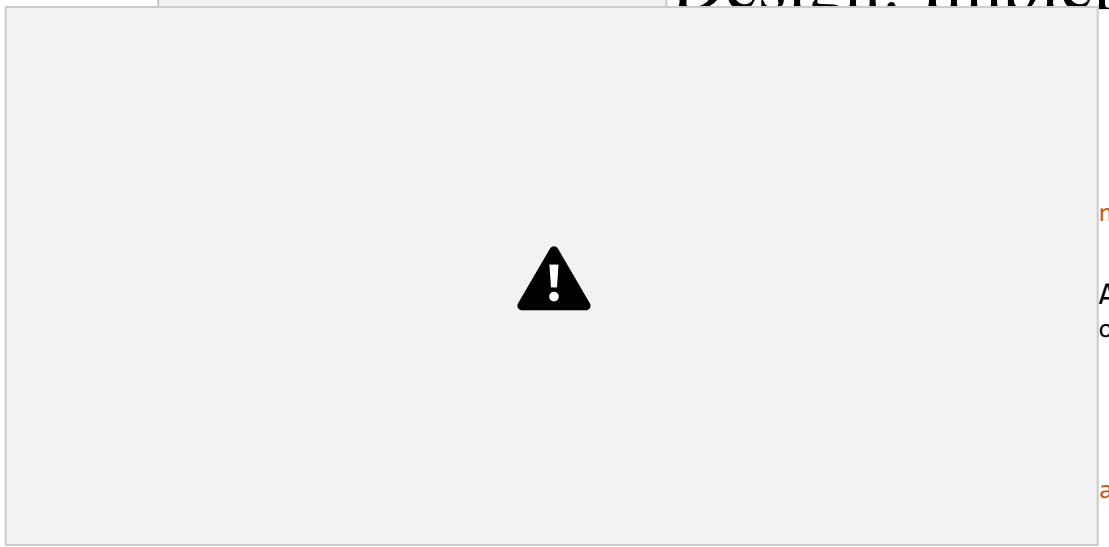
```
set (APIs): */ ...
Not (in = , out = ); And (a = , b =
, out = ); Or (a = , b = , out = );
Xor (a = , b = , out = ); ...
```

HDL *stub file*

```
/** Chips
```



Design: Implementation



a

b

Or

notb

And

out

out

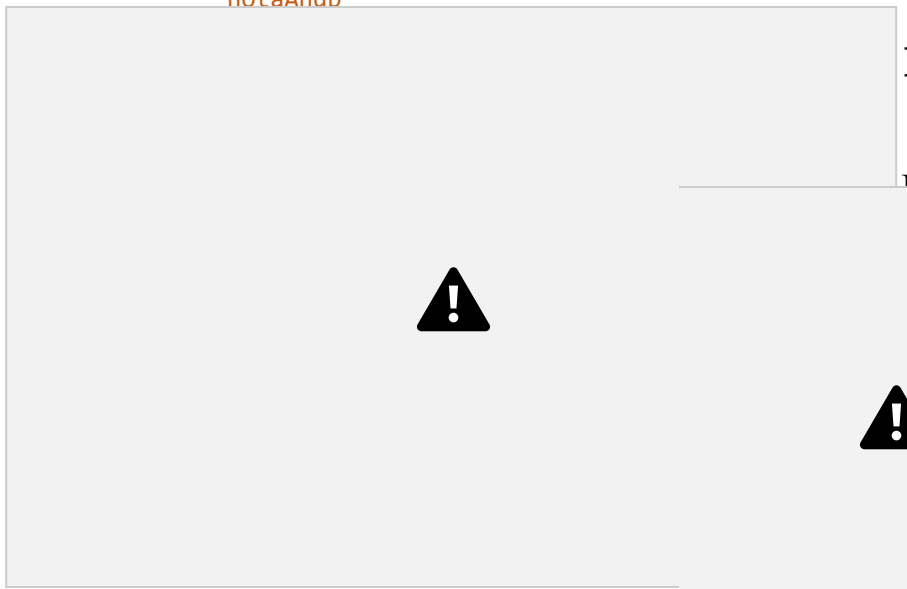
aAndNotb

diagram out

Gate

in nota out
out b
And

b a
Not notaAndb



}

HDL *stub file*

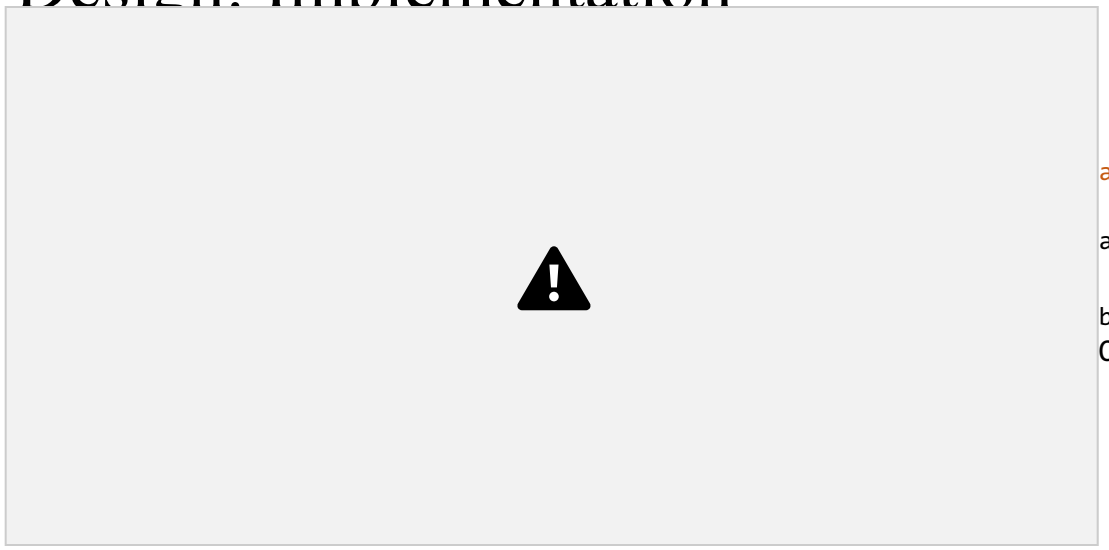
/** Chips set (APIs): */ ... Not (in = , out =); And (a = , b = , out =); Or (a = , b = , out =); Xor (a = , b = , out =); ...

/** out = (a And Not(b)) Or (Not(a) And b)) */ CHIP Xor {

IN a, b;
OUT out;

PARTS:
// Missing implementation

Design: Implementation



Gate diagram

a

b

out

aAndNotb

Or

Not

in

out

Not

in

out

And

a

b

out

And

a

b

out

notb

nota

notaAndb

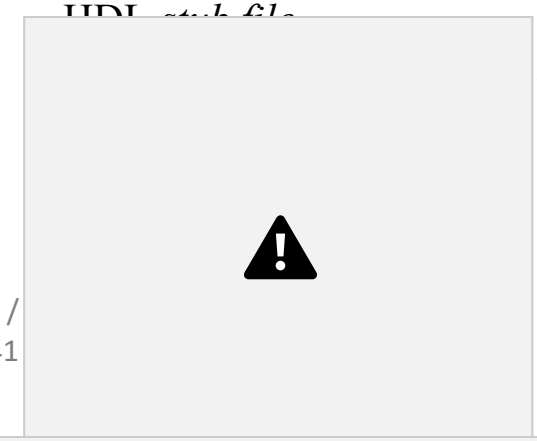
```
CHIP Xor {
 IN a, b;
 OUT out;
 PARTS:
 // Missing implementation
}
```

set (APIs): */ ...

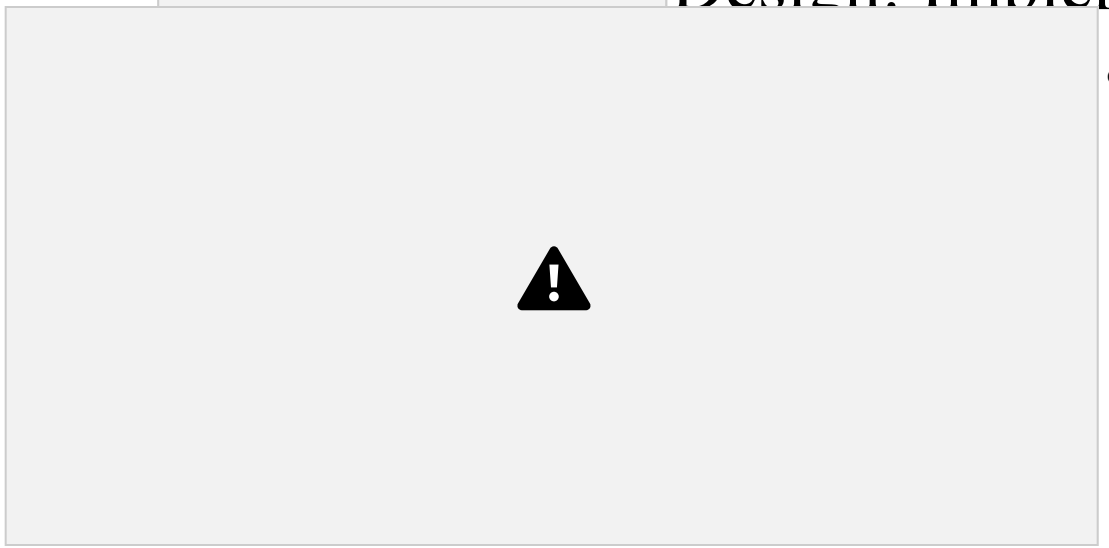
```
Not (in = , out = ); And (a = , b =
, out = ); Or (a = , b = , out = );
Xor (a = , b = , out = ); ...
```

HDL *stub file*

```
/** Chips
```



Design: Implementation



b
Or

aAndNotb

a

out

out

Not

out

in

nota

b

out

And

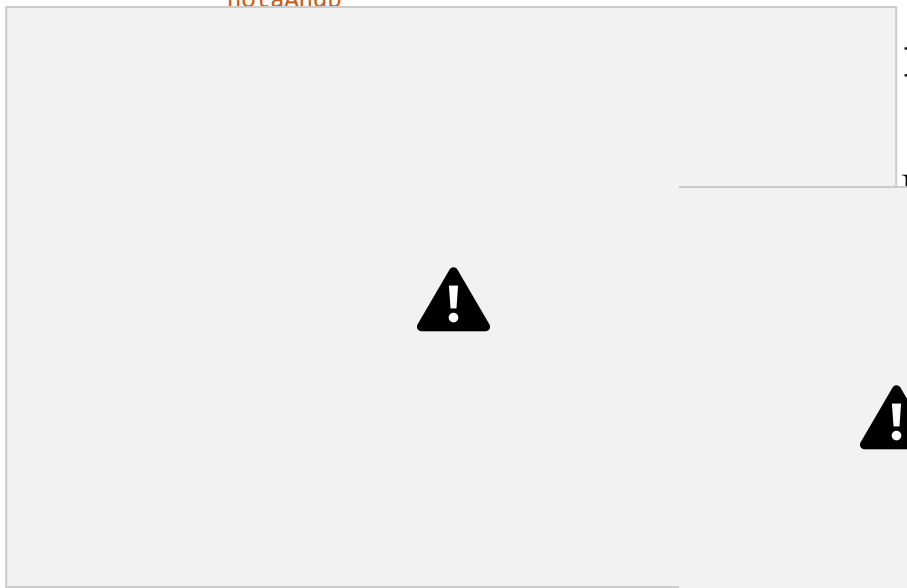
out

notaAndb

b

}

HDL *stub file*



/** Chips set (APIs): */ ...

Not (in = , out =); And (a = , b = , out =);

Or (a = , b = , out =); Xor (a = , b = , out =); ...

/** out = (a And Not(b)) Or (Not(a) And b)) */ CHIP Xor {

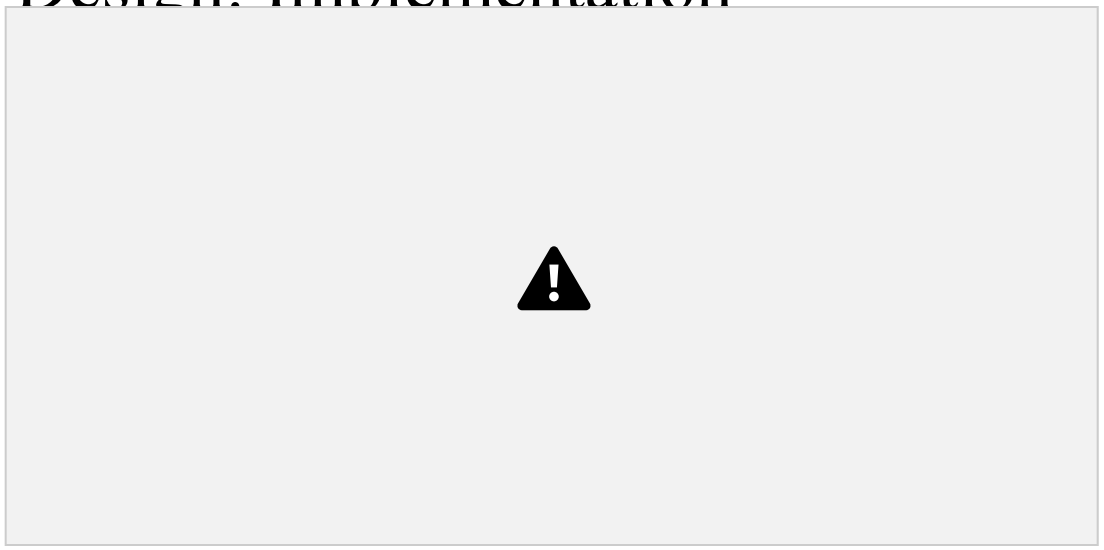
IN a, b;

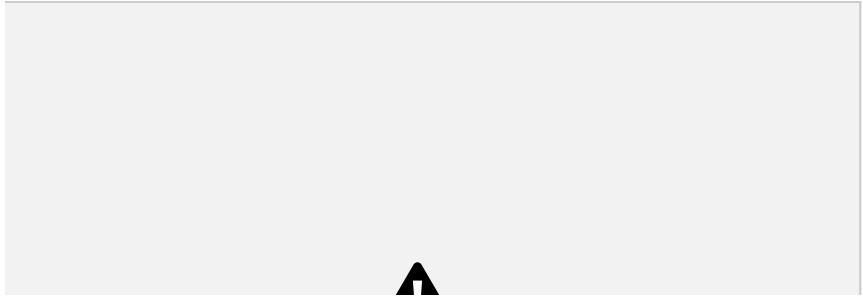
OUT out;

PARTS:

// Missing implementation

Design: Implementation

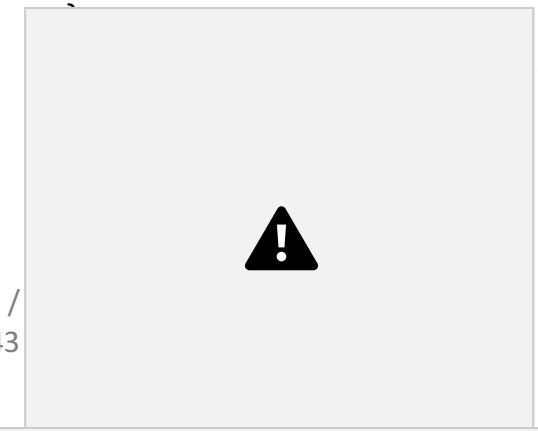




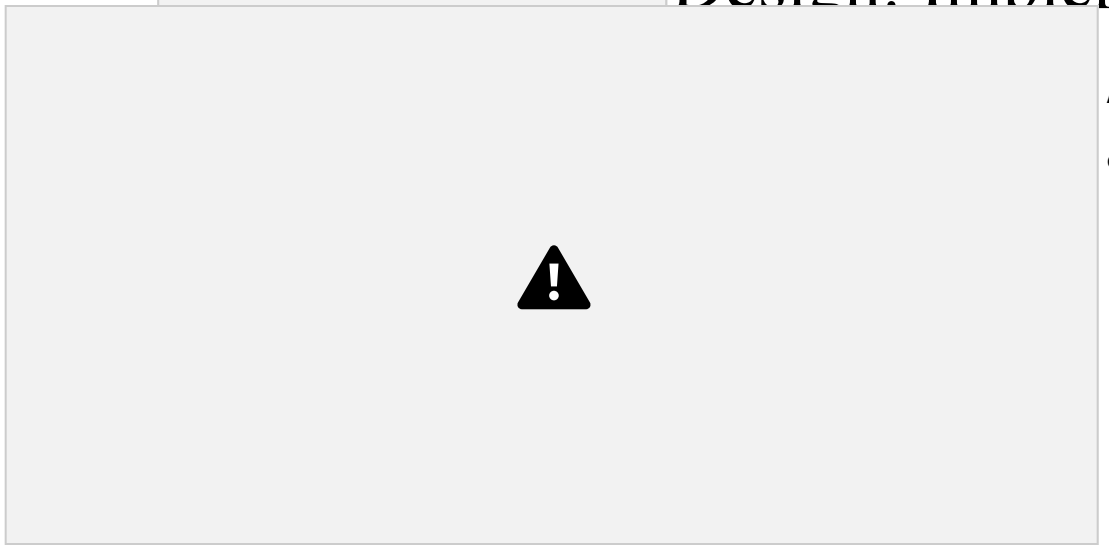
```
Or (Not(a) And b)) */ CHIP Xor {
 IN a, b;
 OUT out;
 PARTS:
 Not (in=a, out=nota);
 Not (in=b, out=notb);
 And (a=a, b=notb, out=aAndNotb);
And (a=nota, b=b, out=notaAndb);
```

```
set (APIs): */ ...
Not (in = , out = ); And (a = , b =
, out = ); Or (a = , b = , out = );
Xor (a = , b = , out = ); ...
```

```
/** Chips
```



Design: Implementation



And

out

a Not in a b out notb

b
Or

aAndNotb

a

out

out

Not

out

in

nota

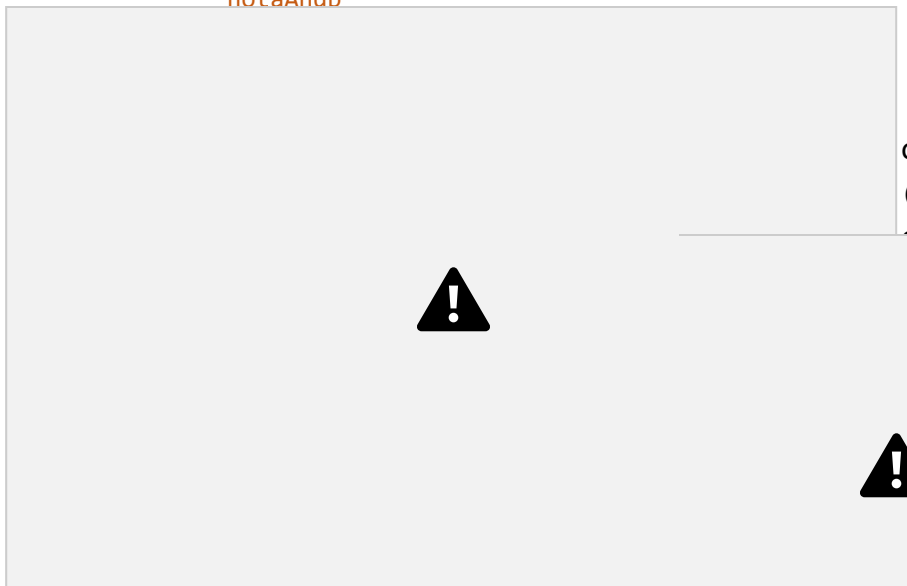
b

out

And

b

notaAndb



Not (in=b, out=notb);
And (a=a, b=notb,
out=aAndNotb); And
(a=nota, b=b,

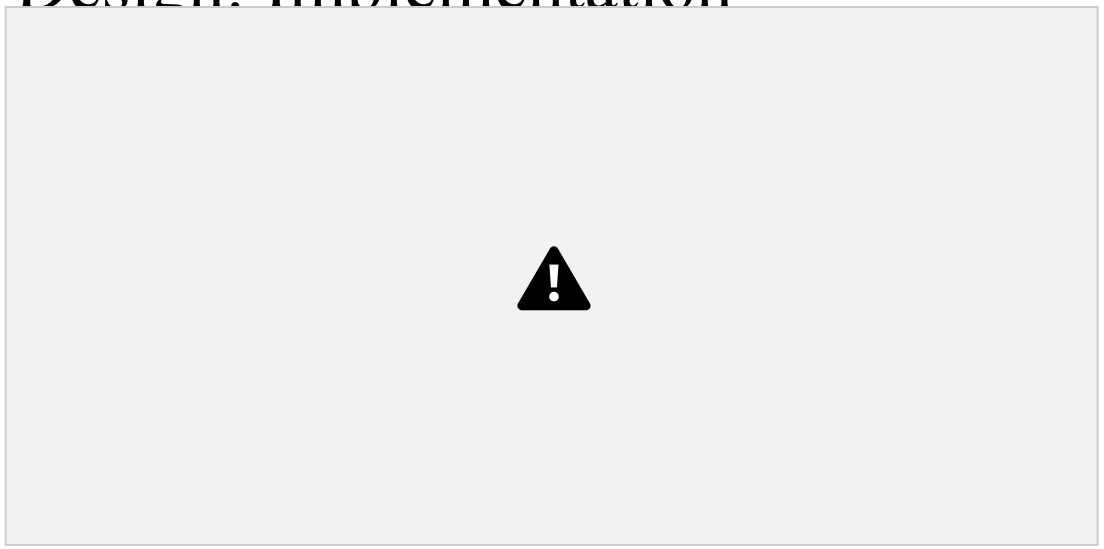
/** Chips set (APIs): */ ...
Not (in = , out =); And (a = , b = , out =);
Or (a = , b = , out =); Xor (a = , b = , out =); ...

/** out = (a And Not(b)) Or (Not(a) And b)) */ CHIP Xor {
IN a, b;
OUT out;
PARTS:
Not (in=a, out=nota);

}

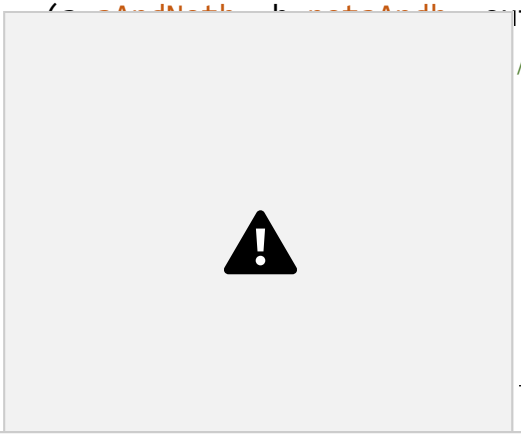
Question: What is the missing HDL line?

Design: Implementation

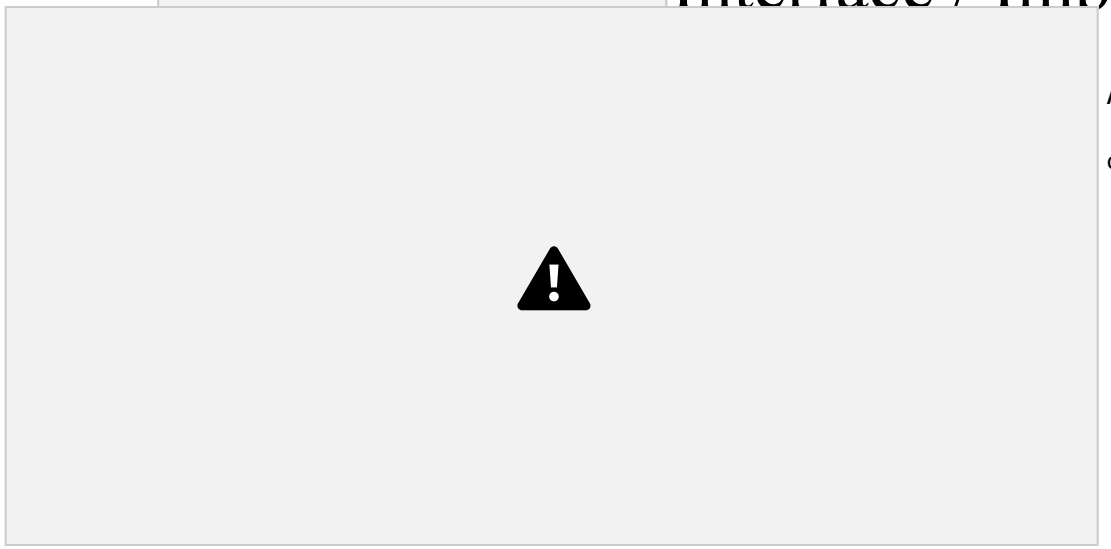


```
Or (Not(a) And b)) */ CHIP Xor {
 IN a, b;
 OUT out;
 PARTS:
 Not (in=a, out=nota);
 Not (in=b, out=notb);
 And (a=a, b=notb, out=aAndNotb);
And (a=nota, b=b, out=notaAndb);  Or
                                  t=out); }
                                  /** Chips
```

```
set (APIs): */ ...
Not (in = , out = ); And (a = , b =
, out = ); Or (a = , b = , out = );
Xor (a = , b = , out = ); ...
```



Interface / Implementation



And

out

b
Or

aAndNotb

a

out

out

out

Not

in

nota

b

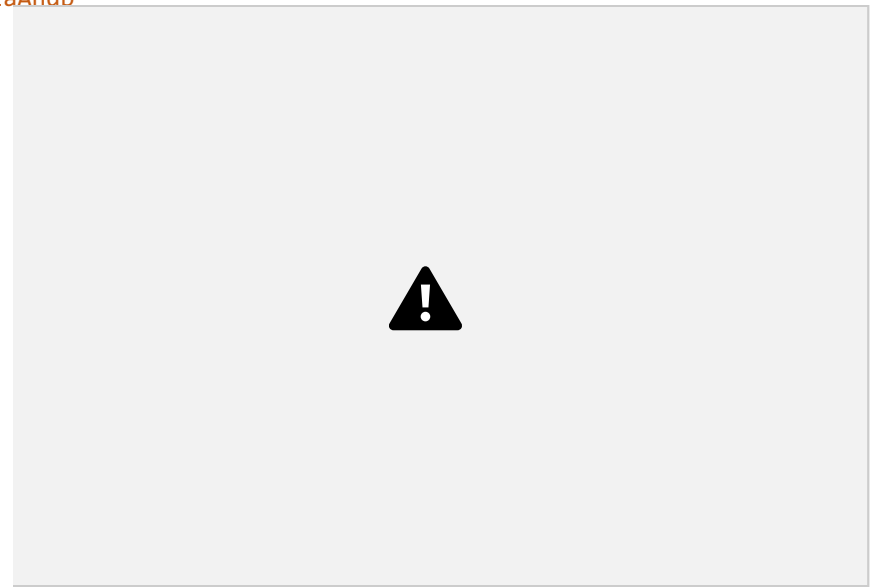
out

And

b

a

notaAndb



And (a=a, b=notb,

gate interface

gate Implement -ation

```
/** out = (a And Not(b)) Or (Not(a) And b)) */ CHIP Xor {
 IN a, b;
 OUT out;
 PARTS:
 Not (in=a, out=nota);
 Not (in=b, out=notb);
 And (a=a, b=notb, out=aAndNotb);  And (a=nota, b=b, out=notaAndb);  Or (a=aAndNotb, b=notaAndb, out=out); }
```

A logic gate has: • One

interface • Many possible implementations

Hardware description languages

Observations

• HDL: a functional / declarative language

• An HDL program can be viewed as a textual / declarative specification of a chip diagram • The order of HDL statements is insignificant.

```
/** out = (a And Not(b)) Or (Not(a) And b)) */
CHIP Xor {
 IN a, b;
 OUT out;
 PARTS:
 Not (in=a, out=nota);
 Not (in=b, out=notb);
```

```
 And (a=a, b=notb, out=aAndNotb);
 And (a=nota, b=b, out=notaAndb);
 Or (a=aAndNotb, b=notaAndb, out=out);
}
```


Hardware description languages

Common HDLs • VHDL

• Verilog

• …

Our HDL

• Captures the essence of other HDLs • Minimal and simple

• Provides all you need for this course • Documentation:

The Elements of Computing Systems / Appendix 2: HDL

```
/** out = (a
And Not(b))
Or (Not(a)
And b)) */
CHIP Xor {
 IN a, b;
 OUT out;
```

```
 PARTS:
 Not (in=a, out=nota);
 Not (in=b, out=notb);
 And (a=a, b=notb, out=aAndNotb);  And
(a=nota, b=b, out=notaAndb);  Or
(a=aAndNotb, b=notaAndb, out=out); }
```


Chapter 1: Boolean logic

Theory

• Basic concepts • Nand

Practice

• Logic gates

• HDL

• Hardware simulation •

Multi-bit buses

Project 1

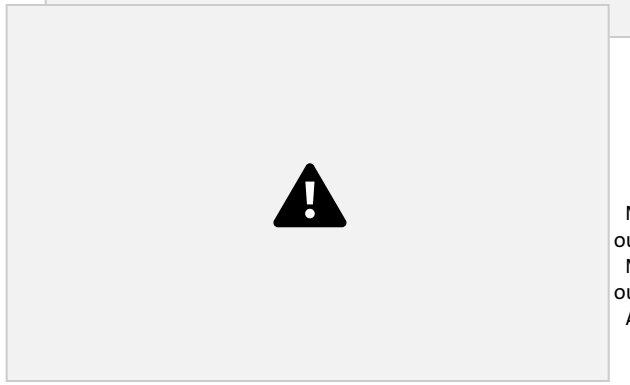
• Introduction • Chips

• Guidelines

Hardware simulation (in a nutshell)

```
CHIP Xor {
```

hardware



simulator

HDL code

```
 Not (in=a,
out=nota);
 Not (in=b,
out=notb);
 And (a=a,
b=notb, out=aAndNotb);  And (a=nota, b=b, out=notaAndb);  Or
(a=aAndNotb, b=notaAndb, out=out); }
```

(desktop / legacy version)

Simulation

options •

Interactive

- Script-based.

1. Load an HDL program

Interactive simulation

3. evaluate the

chip logic 1. Load an HDL
program

2. manipulate
input pins

4. inspect
output pins

HDL code

5. inspect
internal pins

Hardware simulation in a nutshell

```
CHIP Xor {

 IN a, b;  OUT out;
```

HDL code

```
 Not (in=a, out=nota);
 Not (in=b, out=notb);
 And (a=a, b=notb, out=aAndNotb);  And (a=nota, b=b,
out=notaAndb);
```

hardware

```
          load Xor.hdl,
```

test script

simulator

```
 Or (a=aAndNotb, b=notaAndb, out=out);
```

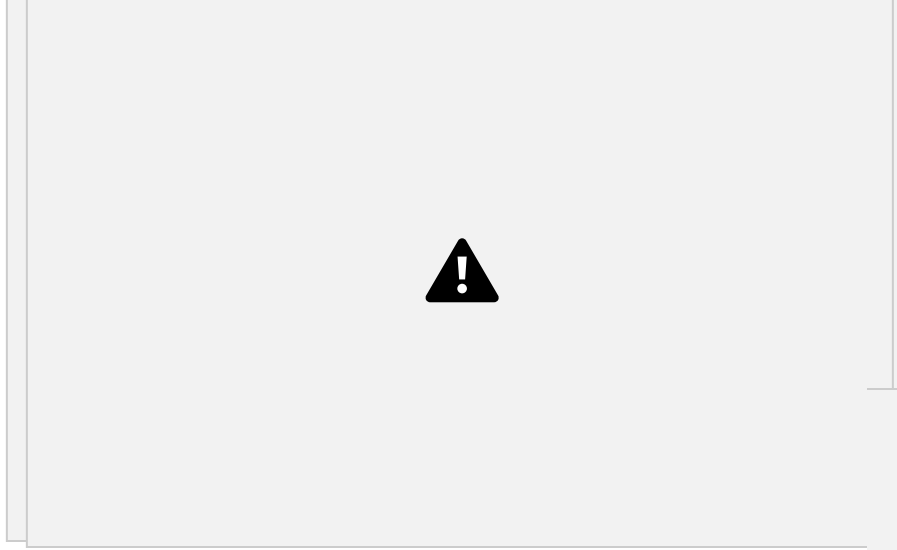

```
            output-file
            And.out,
}
            output-list a b out;
            set a 0, set b 0, eval, output;
            set a 0, set b 1, eval, output;
            set a 1, set b 0, eval, output;
            set a 1, set b 1, eval, output;
```

Simulation options

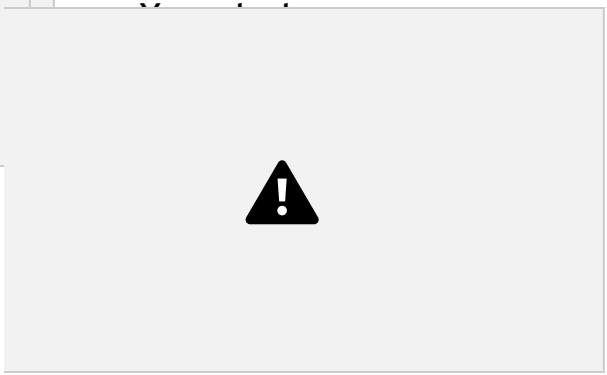
- Interactive
- Script-based.

Script-based simulation

`Xor.hdl`



- “Automatic” testing
- Replicable testing.



```
load Xor.hdl;
set a 0, set b 0, eval;
set a 0, set b 1, eval;
set a 1, set b 0, eval;
set a 1, set b 1, eval;
```

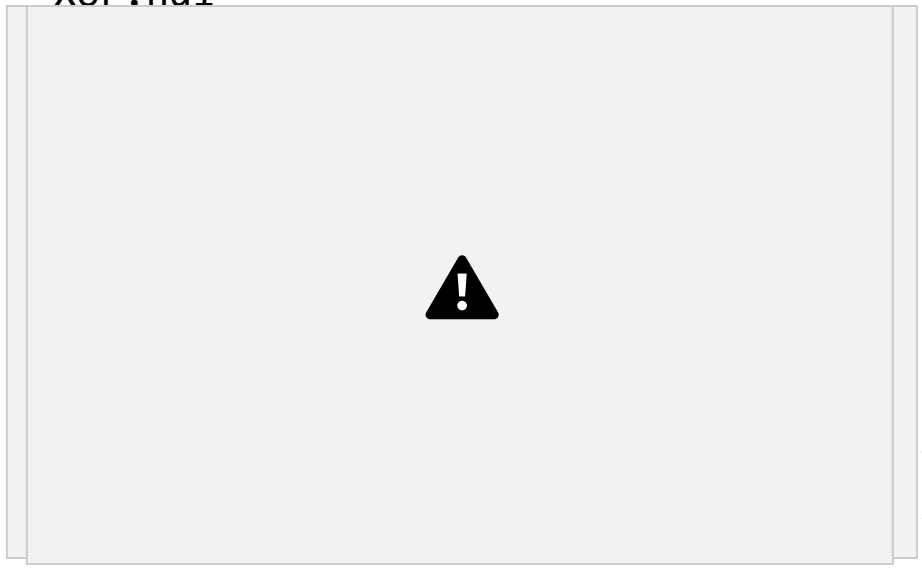
```
CHIP Xor {
 IN a, b;
 OUT out;

 PARTS:
 Not (in=a, out=nota);
 Not (in=b, out=notb);
 And (a=a, b=notb, out=aAndNotb);  And (a=nota, b=b, out=notaAndb);  Or (a=aAndNotb, b=notaAndb, out=out); }
```

test script = sequence of commands to the simulator

Benefits:

Script-based simulation, with an output file

Xor.hdl



```
CHIP Xor {
    IN a, b;
    OUT out;

    PARTS:
    Not (in=a,
out=nota);
Not (in=b,
out=notb);
```

tested chip

Xor.tst

```
load Xor.hdl,
output-file Xor.out,
output-list a b out;
set a 0, set b 0, eval,
output; set a 0, set b 1,
```

```
eval, output; set a 1,    set b 0, eval, output;
   And (a=a, b=notb, out=aAndNotb);  And
  (a=nota, b=b, out=notaAndb);  Or
  (a=aAndNotb, b=notaAndb, out=out);
```

```
set a 1, set b 1, eval, output;
```

values will be written to the output file

```
0 | 1 | | 1 | 1 | 0 |
```

test script

}

The logic of a typical test script

- Initialize:
  - Loads an HDL file
  - Creates an empty output file
  - Lists the names of the pins
- Repeat:
  - `Set` (inputs) , `eval` (chip logic) , `output` (print)

```
| a | b
|out| | 0
| 0 | 0 |
| 0 | 1 |
| | 1 |
```

Output File, created by the test script, as a side-effect of the simulation process

1. Load a

test script

Script-based simulation

2. run
the script

1. Load a test script test script

Script-based simulation

inspect the

2. run output file the

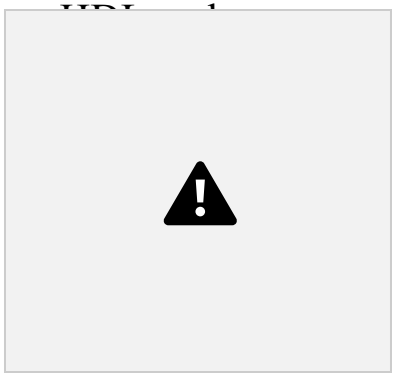
script 1. Load a

test script

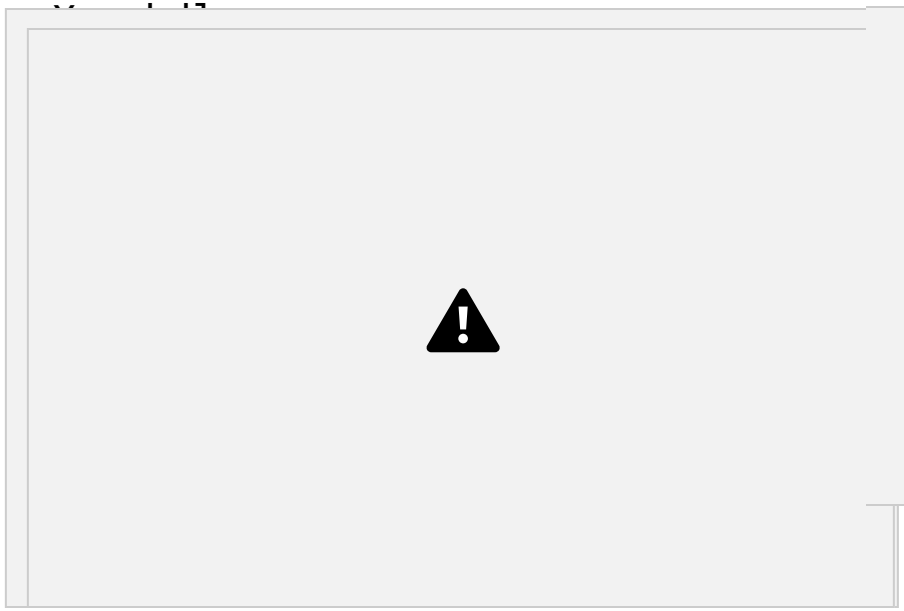
test script

Xor.out

```
| a | b |out| |
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
```

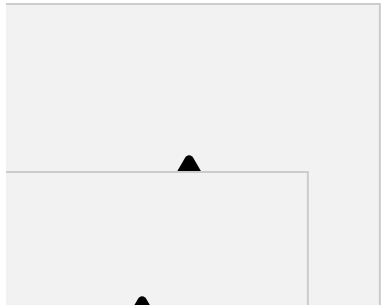


Script-based simulation



load Xor.hdl,
output-file Xor.out,
output-list a b out;
set a 0, set b 0, eval, output; set a 0, set b 1, eval, output; set a 1, set b 0, eval, output; set a 1, set b 1, eval, output;

```
CHIP Xor {
 IN a, b;
 OUT out;

 PARTS:
 Not (in=a, out=nota);
 Not (in=b, out=notb);
 And (a=a, b=notb, out=aAndNotb);  And
(a=nota, b=b, out=notaAndb);  Or
(a=aAndNotb, b=notaAndb, out=out); }
```



Xor.out

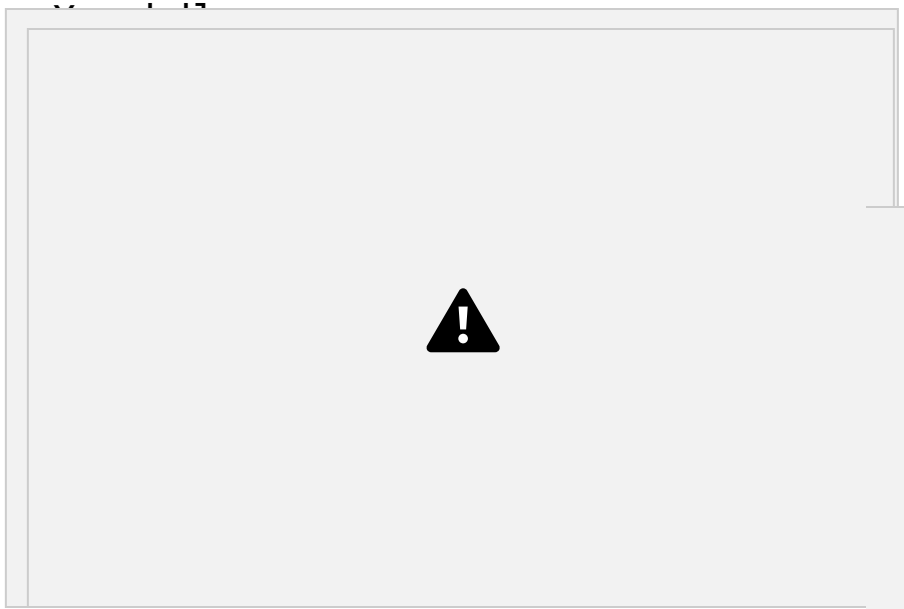
| a | b |out|

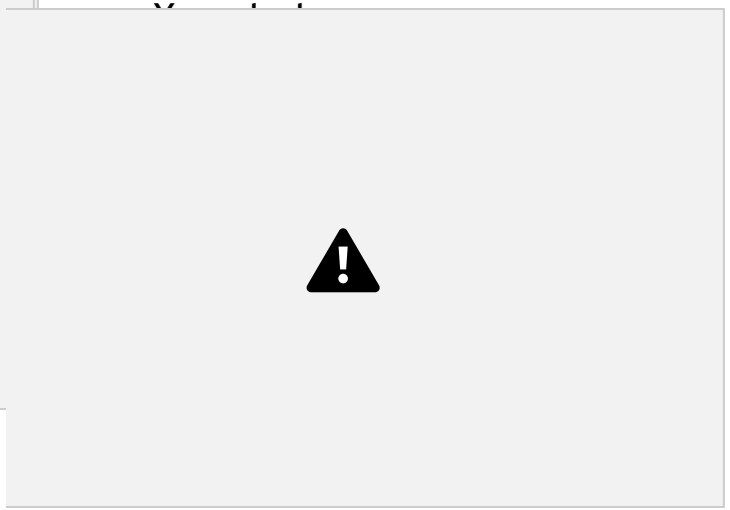
```
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
```

```
| 1 | 1 | 0 |
```


Script-based simulation, with a compare file

```
Not (in=a, out=nota);
Not (in=b, out=notb);
And (a=a, b=notb, out=aAndNotb);  And
(a=nota, b=b, out=notaAndb);  Or
(a=aAndNotb, b=notaAndb, out=out); }
```





load

```
CHIP Xor {
 IN a, b;
 OUT out;

 PARTS:
```

```
Xor.hdl,
output-file Xor.out,
compare-to Xor.cmp,
```

```
output-list a b out;
set a 0, set b 0, eval,
output; set a 0, set b 1,
eval, output; set a 1, set b
0, eval, output; set a 1, set
b 1, eval, output;
```

le logic • When each output command is executed,

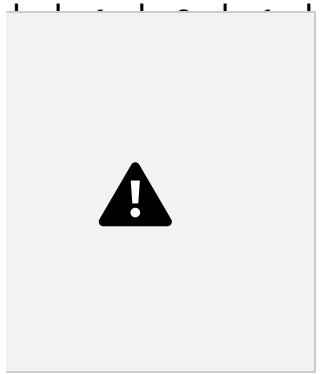
Simulation-with-compare-fi

```
Xor.out

| a | b |out| | 0 | 0 | 0
| 0 | 1 | 1 | | 1 | 0 | 1 |
| 1 | 0 | 1 | | 1 | 1 | 0 |
| 1 | 1 | 0 |
```

```
Xor.cmp

| a | b |out| | 0 | 0 | 0
|
| 0 | 1 | 1 | | 1 | 0 | 1 |
| 1 | 1 | 0 | | 1 | 1 | 0 |
```

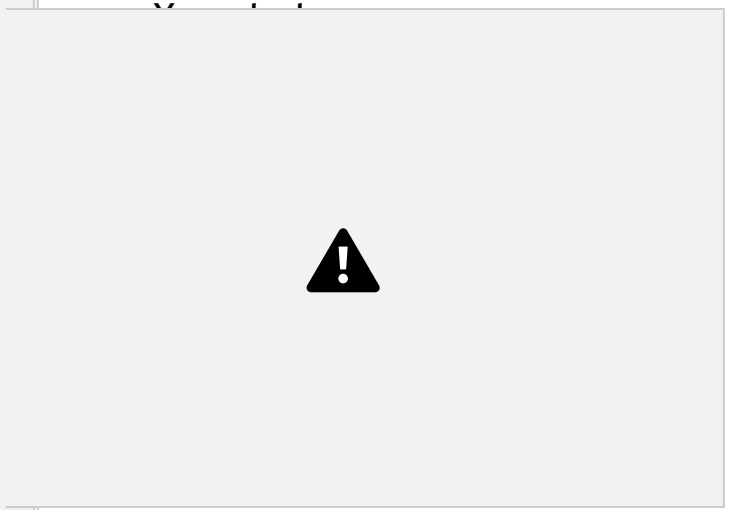
the outputted line is compared to the corresponding line in the compare file

• If the two lines are not the same, the a



Script-based simulation, with a compare file

Xor.hdl



load

Xor.hdl,
output-file Xor.out,
compare-to Xor.cmp,
output-list a b out;
set a 0, set b 0, eval, output; set a 0, set b 1, eval, output; set a 1, set b 0, eval, output; set a 1, set b 1, eval, output;

```
CHIP Xor {
 IN a, b;
 OUT out;

 PARTS:
 Not (in=a, out=nota);
 Not (in=b, out=notb);
 And (a=a, b=notb, out=aAndNotb);  And
(a=nota, b=b, out=notaAndb);  Or
(a=aAndNotb, b=notaAndb, out=out); }
```

[Experimenting with Built-In Chips]

Xor.out

| a | b |out| | 0 | 0 | 0 |

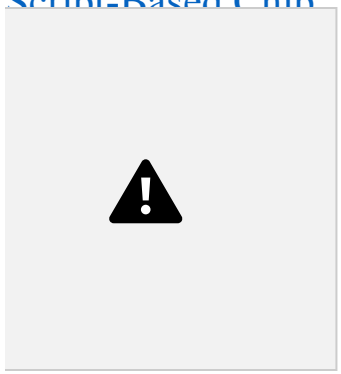
Xor.cmp

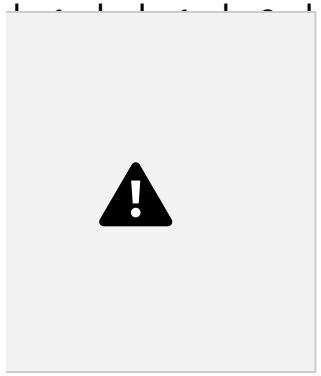
| a | b |out| | 0 | 0 | 0 |

Demos:

Building and Testing

HDL-based Chips





Chapter 1: Boolean logic

Theory

• Basic concepts • Nand

Practice

• Logic gates

• HDL

• Hardware simulation • Multi-bit buses

Project 1

• Introduction • Chips

• Guidelines

Multi-bit bus

• Sometimes we wish to manipulate a *sequence of bits* as a single entity • Such a multi-bit entity is called “bus”

Example: 16-bit bus

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0 **1 0 0 0 1 0 0 0 1 1 0 1 1 1 0 1**

LSB = Least significant bit

MSB = Most significant bit

Working with buses: Examples

```
 IN a[16], b[16]; ...
OUT out[16];      }

 PARTS:
```

16-bit

16
out

15 …
1 0

b[16]

adder

```
/* Adds three
16-bit inputs. */
CHIP Adder3Way {
```

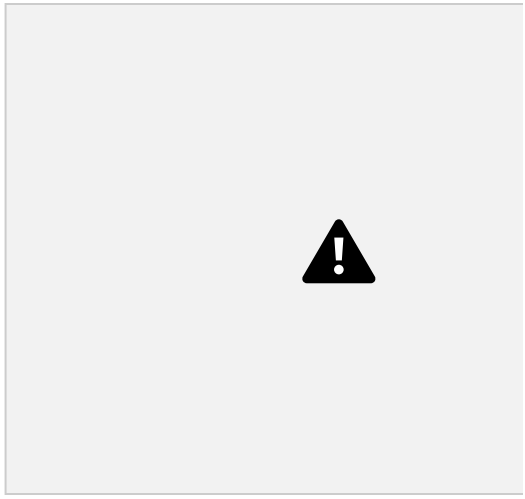
```
a:
1 ...1
```

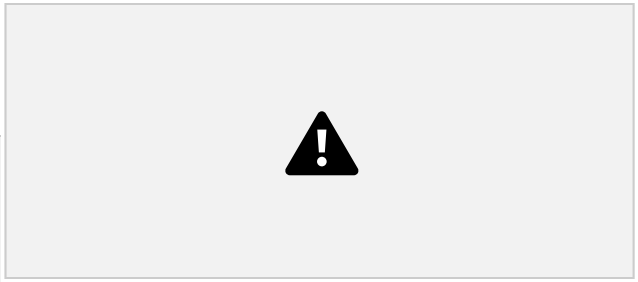
```
 IN a[16], b[16],
c[16];
1
```

```
OUT out[16];
```

```
b: 0 ...1 0 c: 0 ...0 1
```

```
out: 1 ...1 0
```





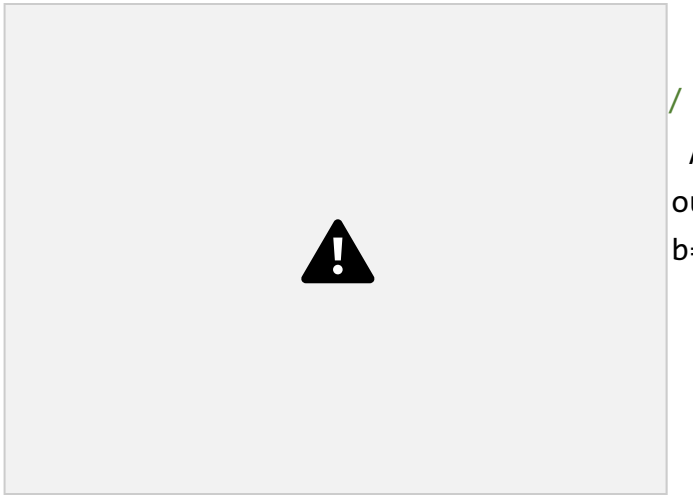
Working with buses: Examples

```
16-bit values
*/
```

```
/* Adds two
```

```
IN a[16], b[16];
OUT out[16];

 PARTS:
 ...
}
```

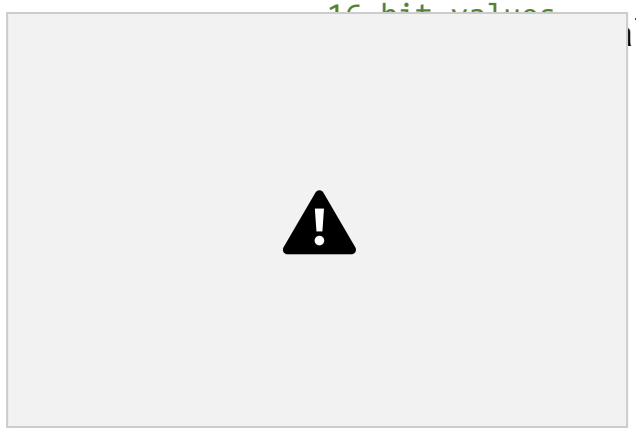
b^{16} 15 …



```
PARTS:
```

```
CHIP Adder3Way {

a:
1 …1
 IN a[16], b[16],
c[16];
1

 OUT out[16];

/

 Adder(a= , b= ,
out= );  Adder(a= ,
b= , out= );  }
```


Working with buses: Examples



16-bit

16
out

```
PARTS:
...
}
```

b^{16}

adder

```
IN a[16], b[16]; OUT out[16];
```

be

```
/*
in
Ad
```

```
OUT out[16];
```

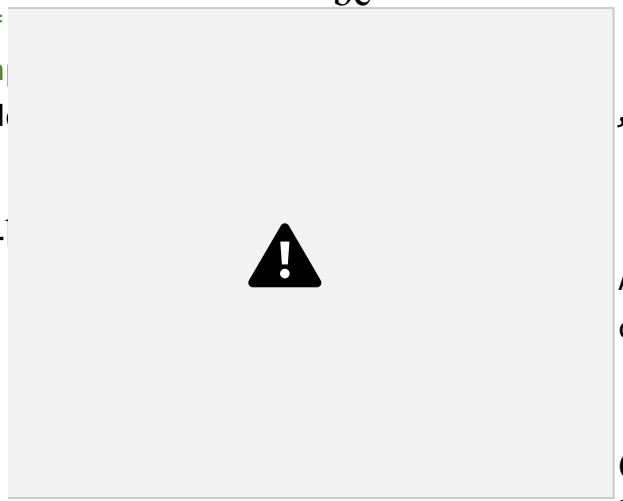
,

treated as a single entity

n-

15 ...

```
b: 0 ...1 0 c: 0 ...0 1

out: 1 ...1 0
```

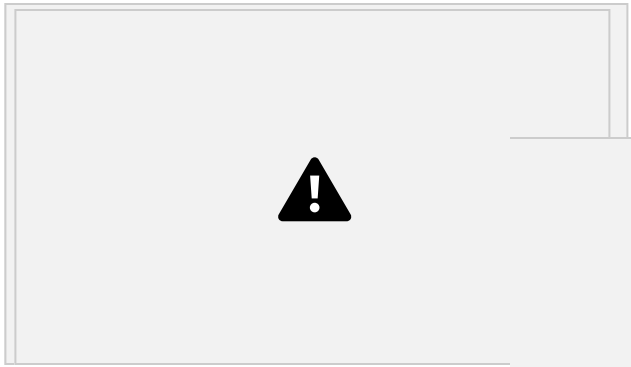


```
Adder(a=ab, b=c,
out=out);  }
```

Creates an internal bus pin (ab)

```
PARTS:
Adder(a=a , b=b, out=ab);
```

Working with individual bits within buses



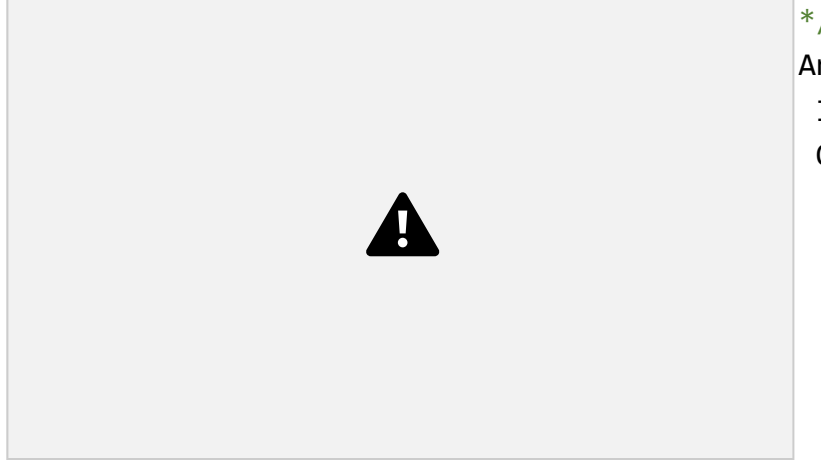
```
 IN a, b;   OUT out;  ...
}
```

```
/* 4-way And: Ands 4 bits.
*/  CHIP And4Way {
 IN a[4];
 OUT out;
```

```
                  1 0
              3 2
a:        0 1 1 1

out:  0
```


Working with individual bits within buses

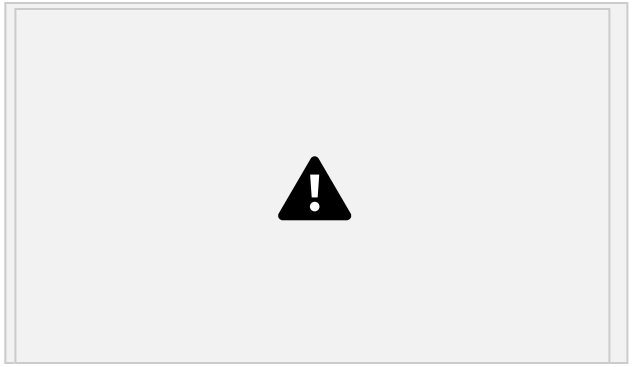
```
/* Returns 1 if a==1 and b=
 0 otherwise. */
CHIP And {
```



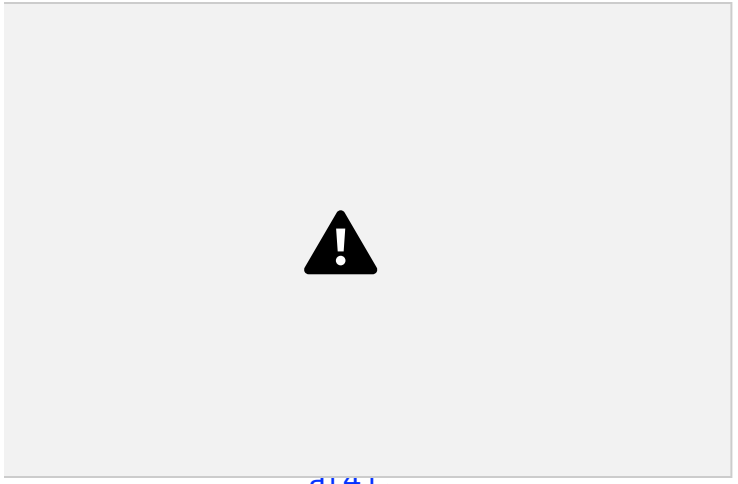
```
*/  CHIP
And4Way {
 IN a[4];
 OUT out;
```

```
/* 4-way And: Ands 4 bits.
                1 0
    3 2          PARTS:
                       ); 1                  );
a:
0 1 1
 And(a= , b= , out=   And(a= , b= , out=
                                    And(a= , b= , out= ); }
        out:
            0
```

Working with individual bits within buses



```
 IN a, b;   OUT out;
...
}
```



```
a[4],
```

```
OUT out;
```

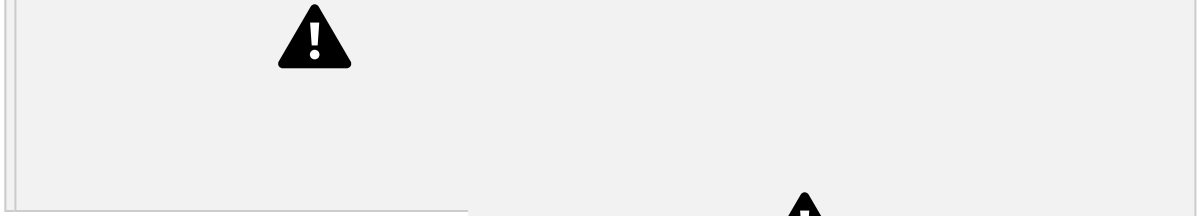
subscripted.

Input bus pins can be

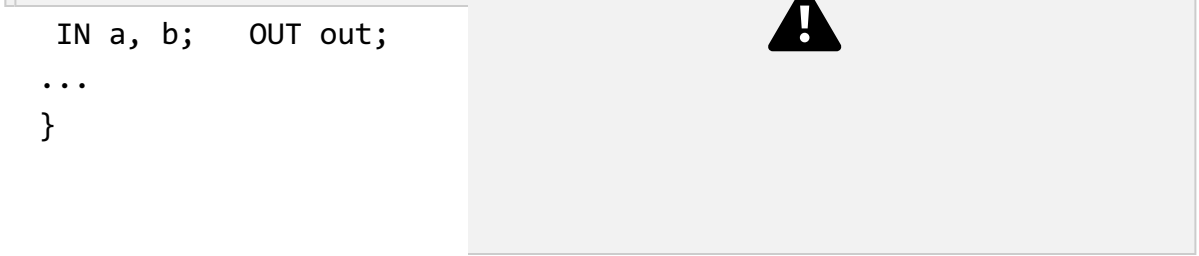
```
                1 0
                 PARTS:
    3 2                 out=and01); 1       b=a[2], out=and012);
a:
0 1 1
 And(a=a[0], b=a[1],  And(a=and01,
                                  And(a=and012, b=a[3],
        out:                     out=out); }
            0
```


Working with individual bits within buses

```
/* Returns 1 if a==1 and b==1,
 0 otherwise. */
CHIP And {
```



```
 IN
a[4];
 OUT
out;
```



```
 IN a, b;   OUT out;
...
}
```

Input bus pins can be subscripted.

```
/* 4-way And: Ands 4
bits. */  CHIP And4Way {
```

```
1 0
```

```
3 2
```

```
 PARTS:
```

```
a:
0 1 1
 And(a=a[0], b=a[1],
out=and01); 1
```

```
3 2
out:
```

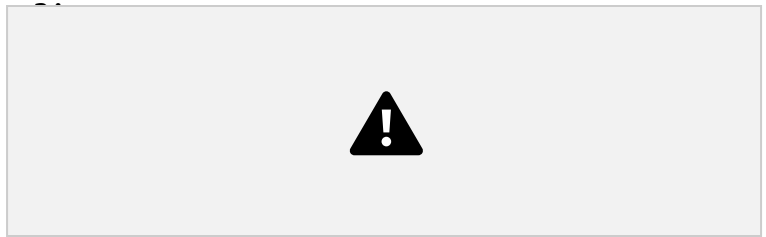
```
 And(a=and01,
b=a[2], out=and012);
```

```
0 1 0
```

```
 And(a=and012,
b=a[3], out=out); }
```

```
/* Bit-wise And of
two 4-bit inputs */
```

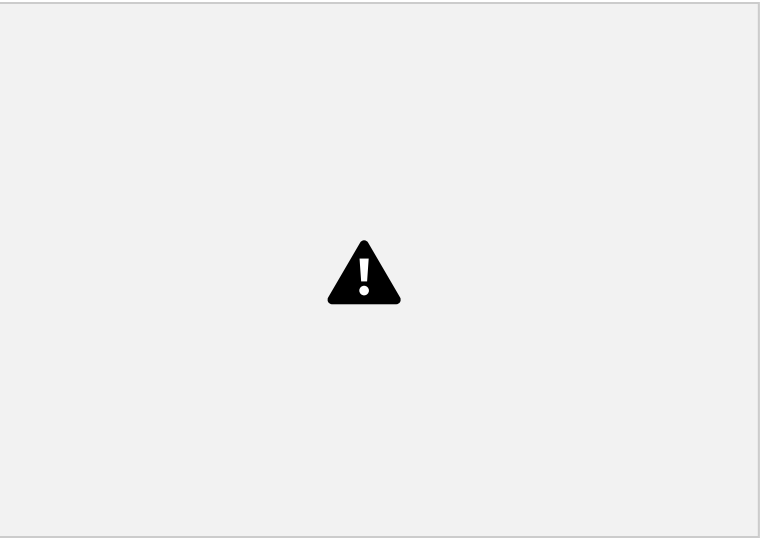
```
1            b[4];
```



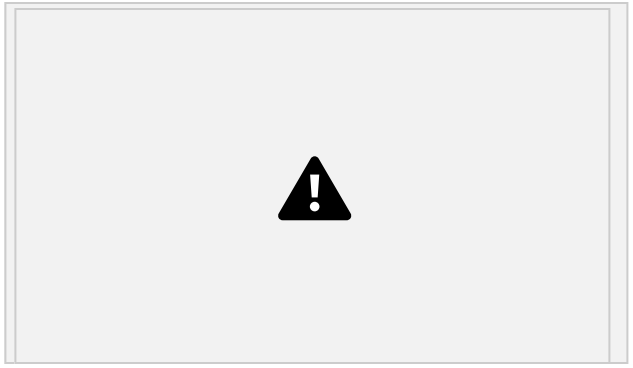
```
CHIP And4 {    IN a[4],
```

```
OUT out[4];
```

```
  b: 0 0 1 1

out: 0 0 0 1
```

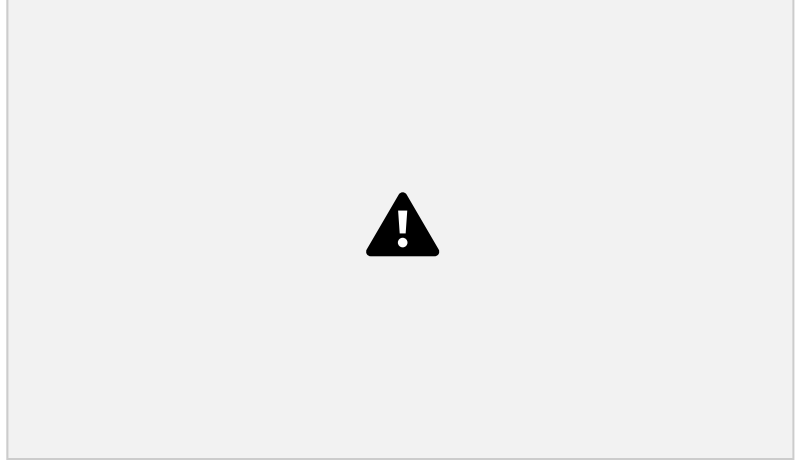


Working with individual bits within buses



```
}
```

```
  IN a, b;   OUT out;
...
```



bits.
*/
CHIP
And4Way
{
 IN
a[4];
 OUT
out;

bus pins can be subscripted.

Input

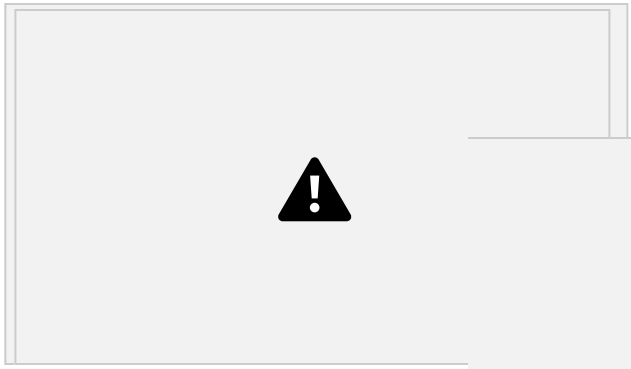
```
/* 4-way And: Ands 4
                1 0
    3 2          PARTS:

a:
0 1 1
 And(a=a[0], b=a[1],
out=and01); 1
                        3 2
                        out:
 And(a=and01,                01 0
b=a[2], out=and012);


                CHIP And4 {    IN a[4],
a:              1             b[4];
0 1 0

b: 0 0 1 1 out: 0 0 0 1
```

```
 And(a=and012,
b=a[3], out=out); }

/* Bit-wise And of
two 4-bit inputs */
```

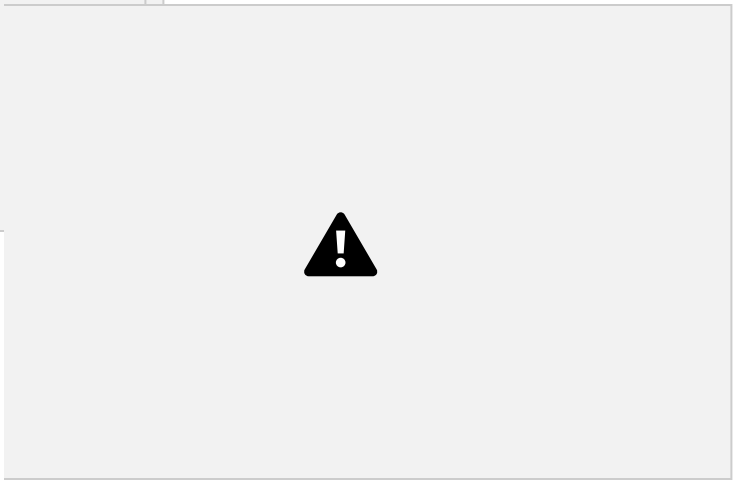
```
 PARTS:
 And(a= , b= ,
out= );  And(a= ,
b= , out= );
And(a= , b= ,
out=. );  And(a= ,
b= , out= ); }
```

```
OUT out[4];
```


Working with individual bits within buses



```
 IN a, b;   OUT out;
...
}
```

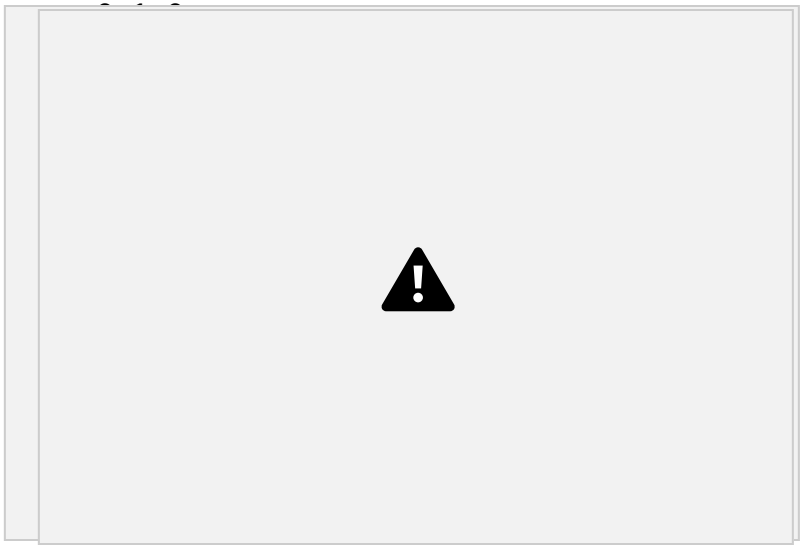


```
 IN
a[4];
 OUT
out;
```

```
/* 4-way And: Ands 4
bits. */  CHIP And4Way {
```

Input bus pins can be subscripted.

```
1 0
 PARTS:
3 2
                                  And(a=and012,
a:                               b=a[3], out=out); }
0 1 1
 And(a=a[0], b=a[1],             /* Bit-wise And of
out=and01); 1                    two 4-bit inputs */
                    3 2
                    out:
 And(a=and01,            0 1 0
b=a[2], out=and012);

                    CHIP And4 {
a:                  1
                                  IN a[4], b[4];
```



```
                                  PARTS:
                                  And(a=a[0], b=b[0],
                                 out=out[0]);
                                 And(a=a[1], b=b[1],
                                 out=out[1]);
                                 And(a=a[2], b=b[2],
                                 out=out[2]);
                                 And(a=a[3], b=b[3],
                                 out=out[3]); }
OUT out[4];
```

Output bus pins

can be subscripted

Additional essential tips:

The Elements of Computing Systems / Appendix 2: HDL

Chapter 1: Boolean logic

Theory

• Basic concepts • Nand to Tetris

• Logic gates

• HDL

• Hardware simulation • Multi-bit buses

Practice

Project 1

• Introduction • Chips

• Guidelines

Chapter 1: Boolean logic

Theory

• Basic concepts • Nand to Tetris • Logic gates • HDL • Hardware simulation • Multi-bit buses

Practice

Project 1

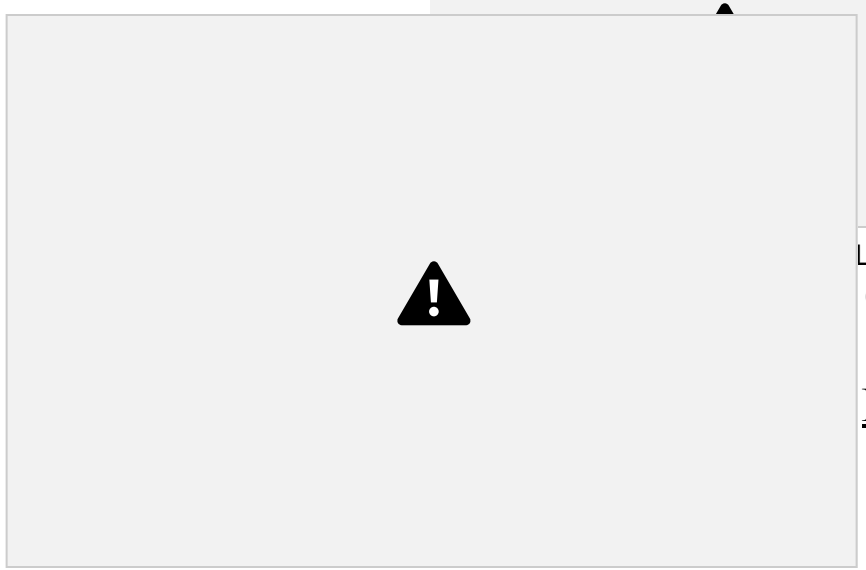
• Introduction • Chips • Guidelines

Built-in chips

We provide built-in versions of the chips built in this course (in `tools/builtInChips`).
For example:

Xor.hdl

```
/** Sets out to a Xor b
*/ CHIP Xor {
 IN a, b;  OUT out;
```



```
 Not (in=a, out=nota);
 Not (in=b, out=notb);
 And (a=a, b=notb, out=aAndNotb);  And
(a=nota, b=b, out=notaAndb);  Or
```

Xor.hdl

```
Sets out to a Xor b
HIP Xor {
```

Internal implementation

```
LTIN Xor;
(a=aAndNotb, b=notaAndb, out=out); }
```

Behavioral simulation

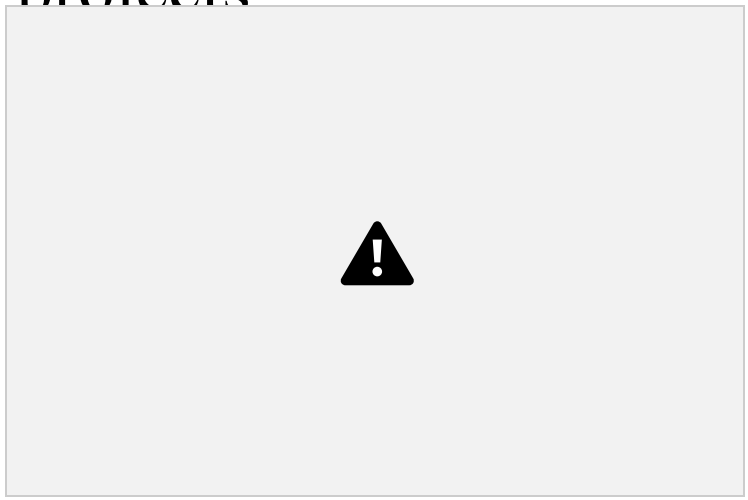
```
// Implemented in Java / Javascript / ... }
```

A built-in chip has the same interface as the regular chip, but a different implementation

• Before building a chip in HDL, one can implement the chip logic in a high-level language • Enables experimenting with / testing the chip abstraction before actually building it • Enables high-level planning and testing of hardware architectures.

Demo: Loading and testing a built-in chip in the hardrawe simulator

Hardware construction projects



Key players:

Architect
- Decides which chips are needed
- Specifies the chips

Developers
- Build / test the chips

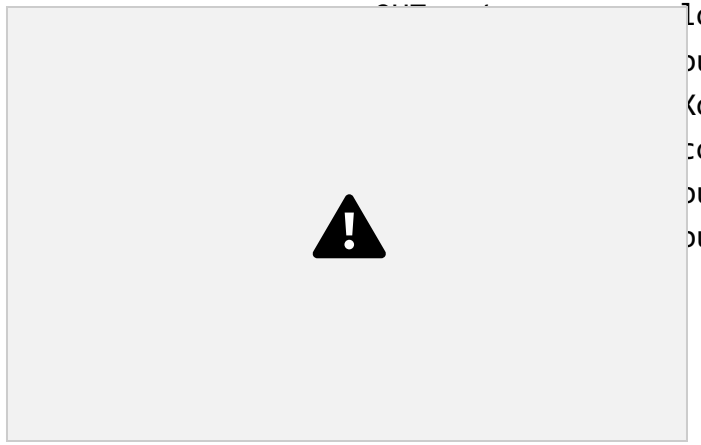
In Nand to Tetris:

The architect is the course team; the developers are the students

For each chip, the architect supplies:
- Chip API (skeletal HDL program = stub file)
- Test script
- Compare file
- Built-in chip

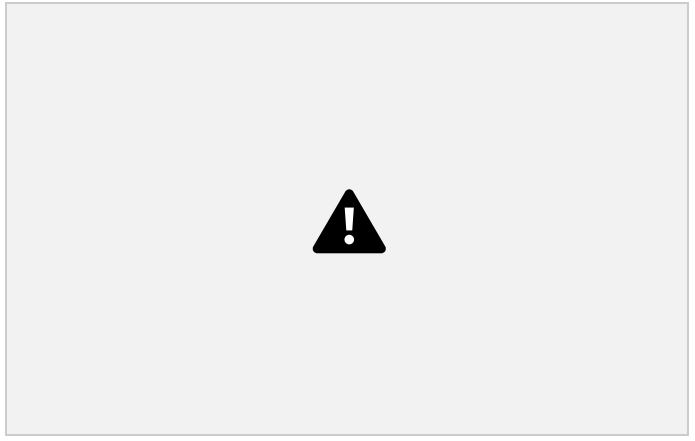
Given these resources, the developers (students) build the chips.

The developer's view (of, say, a `Xor` gate)



```
IN a, b;
                                load Xor.hdl,
                                output-file
                                Xor.out,
                                compare-to Xor.cmp
                                output-list a b
                                out;
                                      output;  set  a  0,  set  b  1,
                                      eval,  output;  set  a  1,  set  b
                                      0,  eval,  output;  set  a  1,  set
                                      b  1,  eval,  output;
PARTS:
// Implementation missing }
```

test script



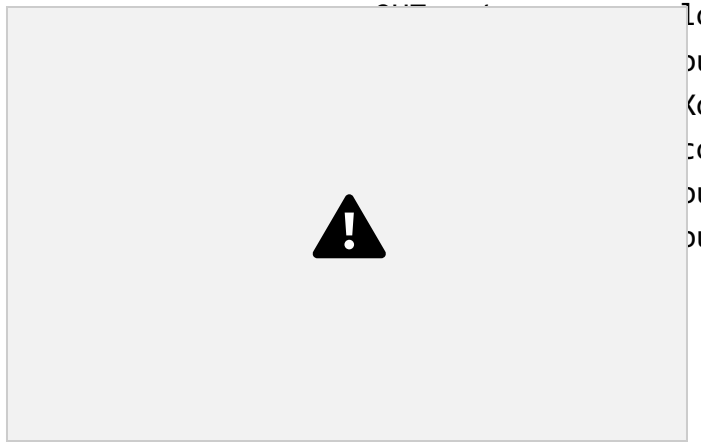
```
set  a  0,  set  b  0,  eval,
```

compare file `Xor.cmp`

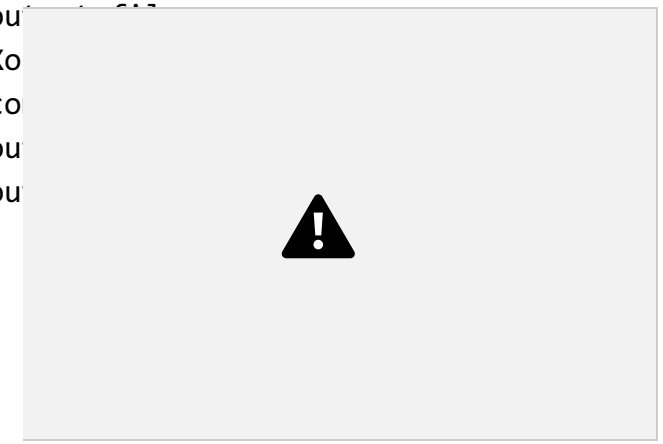
```
| a | b |out|
| 0 | 0 | 0 | | 0 | 1 | 1 | | 1 | 0 | 1 | | 1 | 1 | 0 |
```

- How the chip is supposed to behave (`.cmp`)
- How to test the

The developer's view (of, say, a `Xor` gate)



```
IN a, b;
load Xor.hdl,
PARTS:
// Implementation missing }
```



```
set a 0, set b 0, eval,
output; set a 0, set b 1,
eval, output; set a 1, set b
0, eval, output; set a 1, set
b 1, eval, output;
```

These files specify:

- The chip interface (`.hdl`)

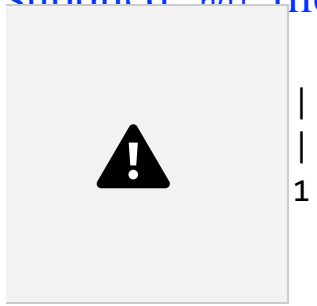
compare `Xor.cmp`

file

```
| a | b |out|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |
```

Implement the chip (complete the supplied `.hdl` file), using these

- How the chip is supposed to behave (`.cmp`)
- How to test the chip (`.tst`)



The developer's task:

Project 1

abstraction

machine language

assembler

abstraction hardware platform

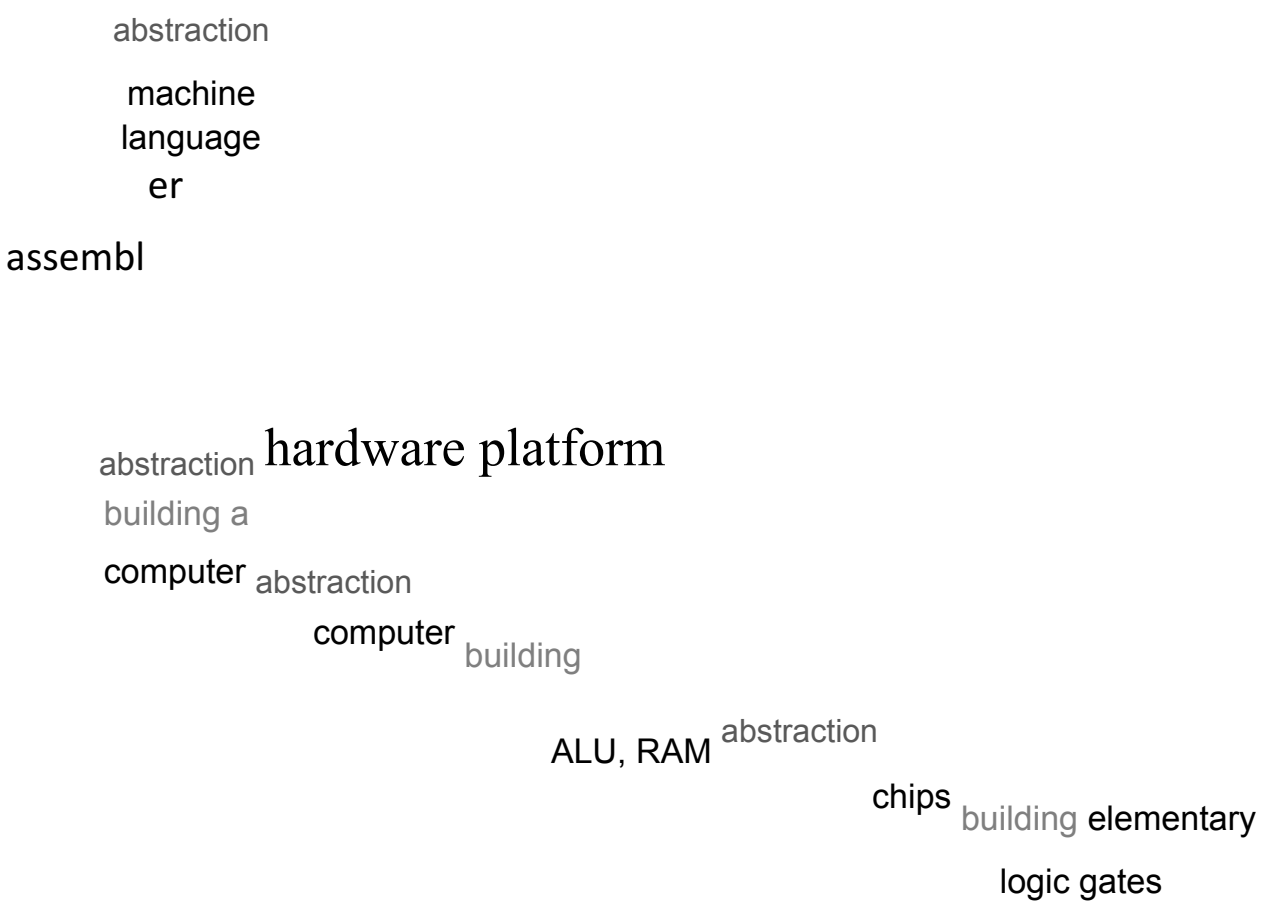
building a computer abstraction

computer building

ALU, RAM abstraction

chips building elementary logic gates gates

Project 1


abstraction
machine
language
er
assembl
abstraction
hardware platform
building a
computer
abstraction
computer
building
ALU, RAM
abstraction
chips
building
elementary
logic gates

gates

Project 1

Build 15 elementary logic gates